

MIDTERM ESSAY

COURSE: WEB PROGRAMMING & APPLICATIONS - 503073

SEMESTER 2 - ACADEMIC YEAR 2025-2026

Version 2.0, last updated by Dr. Van-Vang Le

I. GENERAL INFORMATION

The midterm project must be completed within 3-4 **weeks**. Selected groups will be required to present their projects and system architectures in class (PowerPoint slides must be prepared for presentations).

There are no rigid constraints on the development of the demonstration website; the more advanced techniques you apply, the higher your evaluation will be. For instance, implementing an AI-integrated feature or a multi-agent backend will be rated significantly higher than a traditional CRUD application.

II. LIST OF TOPICS

Note: The following table provides suggestions. Completing all criteria does not guarantee a perfect score. You are encouraged to explore advanced technical layers.

ID	Topic Name	Description & Demo Suggestions
1	Advanced Identity & Access Management (IAM)	Key Points: OAuth2.0, JWT, biometric authentication (WebAuthn), Role-Based Access Control (RBAC). Demo: Implement a login system using Google/GitHub OAuth, integrating 2FA (Authenticator App), and robust password hashing (Argon2/bcrypt).
2	Modern Front-End Ecosystems (React/Next.js)	Key Points: Server-Side Rendering (SSR) vs. Static Site Generation (SSG), state management (Redux/Zustand), TailwindCSS

ID	Topic Name	Description & Demo Suggestions
		vs. Material-UI. Demo: Build an e-commerce dashboard utilizing Next.js for SEO optimization, incorporating global state management for a shopping cart.
3	API Design: RESTful vs GraphQL	Key Points: Over-fetching/under-fetching solutions, query vs. mutation, stateless architecture. Demo: Develop a dual-API backend (providing both REST and GraphQL endpoints) for a library system, and demonstrate fetching efficiency on the frontend.
4	Real-Time Web: WebSockets & Server-Sent Events	Key Points: Bi-directional communication, event-driven architecture, <u>Socket.io</u> vs. native WebSockets. Demo: Create a collaborative real-time workspace (e.g., shared whiteboard or live document editor) with instant state synchronization across multiple clients.
5	Progressive Web Apps (PWAs)	Key Points: Service Workers, caching strategies, offline-first development, Web App Manifest. Demo: Develop a task-tracking PWA that functions entirely offline, syncing data to the server once the network is restored. Include push notifications.
6	Web Security & Vulnerability Mitigation	Key Points: OWASP Top 10, XSS, CSRF, SQL Injection prevention, Content Security Policy (CSP). Demo: Build a "vulnerable" web app alongside a "hardened" version. Demonstrate an active exploit on the vulnerable site and how the hardened site prevents it.
7	WebAssembly (Wasm) for High Performance	Key Points: Bridging C/C++/Rust with JS, browser execution sandbox, heavy computation offloading. Demo: Compile an image-processing algorithm

ID	Topic Name	Description & Demo Suggestions
		from C++ to Wasm and run it in the browser, comparing its execution speed with a pure JavaScript equivalent.
8	Intelligent Web Apps (AI/LLM Integration)	Key Points: Utilizing OpenAI APIs, Hugging Face, or browser-based ML (TensorFlow.js) directly in web clients. Demo: Build a smart customer service bot or an automated document summarizer using LangChain.js or TensorFlow.js running natively in the browser.
9	Cloud Deployment & CI/CD Pipelines	Key Points: Automated testing, GitHub Actions, Vercel/Netlify, AWS EC2, SSL/TLS certificates. Demo: Set up a fully automated CI/CD pipeline where pushing code to the <code>main</code> branch automatically runs tests and deploys to a production server with HTTPS enabled.
10	Containerization & Orchestration (Docker)	Key Points: Dockerfiles, Docker Compose, microservices, container isolation. Demo: Containerize a multi-tier web application (Frontend, Backend API, Redis, and PostgreSQL) and run it using a single <code>docker-compose up</code> command.

III. GENERAL REQUIREMENTS

1. Research and Architecture:

- Conduct a thorough investigation into the assigned topic.
- Clearly define the **System Architecture** before writing code. Utilize UML diagrams or modern architecture charts.
- Use reliable academic and official documentation sources. Include examples, strengths, weaknesses, and a comparative analysis with alternative technologies.

2. Demonstration Website:

- Develop a fully functional website applying the assigned technology.
- The application should follow modern UI/UX principles.
- Ensure the project structure is logical and strictly follows the MVC or component-based paradigms.

3. Writing and Presentation:

- The report must be structured logically: Introduction, Theoretical Survey, Requirement Analysis, System Design, Results & Discussion, and Conclusion.
- Utilize proper grammar, IEEE citation format, and professional technical language.

IV. REQUIREMENTS FOR OUTPUT

1. **Complete Report (Word & PDF):** Must adhere to the Faculty of Information Technology's official template. Avoid AI-generated padding; focus on concise, technical analysis.
2. **Source Code & Repository:** Provide a link to a public/private GitHub repository containing the complete, well-documented source code and dependency file.
3. **Product Presentation Video:**
 - **Duration:** Maximum 15 minutes.
 - **Content:** Cover theoretical aspects, architectural design, and a live demonstration of the codebase and application.
 - **Participation:** All team members must actively speak and present.
4. **README.md:** Include a markdown file in the root directory with step-by-step setup instructions, environment variables template, and testing credentials.

V. DETAILED SCORING (10.0 TOTAL)

CRITERIA	POINT	0%	50%	100% (Full Score)
REPORT	5.0			

CRITERIA	POINT	0%	50%	100% (Full Score)
Theoretical & Architecture	1.5	No relevant content.	Basic theory, missing architecture diagrams.	In-depth explanation with professional architectural design diagrams.
Comparative Analysis	1.0	No comparisons made.	Vague comparison, lacks technical depth.	Insightful comparison of alternative technologies highlighting pros/cons.
Implementation Detail	1.5	No explanation of code.	Brief explanation, missing core logic details.	Clear, step-by-step explanation of how the system was built and deployed.
Structure & Formatting	1.0	Critical formatting errors.	Minor formatting issues.	Professional formatting, excellent use of charts, tables, and correct citations.
DEMO	5.0			
Scope & Complexity	3.0	Non-functional demo.	Basic functionality, simplistic approach.	Comprehensive, complex implementation addressing all topic requirements perfectly.
Video & Presentation	2.0	No video submitted.	Poor quality, unequal speaking time.	High-quality explanation, clear audio/video, equal team participation.

Penalty Policy

- **Late Submission:** 1.0 point deducted per day late.
- **Team Size:** Must be exactly 2-3 members.
- **Language:** Non-English reports/videos will incur a 1.0 point deduction.
- **Missing Instructions:** Failure to provide a proper `README.md` or admin credentials will result in an immediate 1.0 point deduction.